

Lời giải HDU Multi-University Training Contest 8 năm 2021

Nguồn gốc: 2021 HDU Multi-University Training Contest 8 solution

contest/hdu-2021-training-08-solutions.pdf

1001 X-liked Counting

Tính trực tiếp tổng số trường hợp mà trong tiền tố hoặc hậu tố có ít nhất một đoạn thỏa điều kiện không thật thuận tiện. Ta xét bù bằng nguyên lý bao hàm: trước hết tính số số mà không có tiền tố hay hậu tố nào thỏa điều kiện, rồi lấy tổng số trường hợp trừ đi lượng này.

Do tiền tố và hậu tố có phần chồng lên nhau, ta không thể chuyển trạng thái đồng thời từ hai đầu. Có thể trước hết liệt kê kết quả của toàn bộ số theo modulo x , sau đó chọn một hướng, từ trước ra sau hoặc từ sau ra trước, để chuyển trạng thái, đồng thời ghi lại kết quả modulo x của tiền tố hoặc hậu tố hiện tại. Như vậy kết quả modulo x của cả tiền tố lẫn hậu tố đều có thể được biểu diễn. Khi chuyển trạng thái, cần chú ý chi tiết không được mang số 0 đứng đầu.

Tổng số trạng thái có thể hơi lớn, nhưng nhiều trạng thái thật ra vô dụng; chỉ cần thêm vài cắt tỉa nhỏ là có thể thông qua.

1002 Buying Snacks

Đặt $dp_{i,j}$ là số phương án sau khi đã xét xong i món ăn vật đầu tiên và tổng cộng dùng j đồng. Ta thu được chuyển trạng thái độ phức tạp $O(n^2)$:

$$dp_{i,j} = dp_{i-1,j} + dp_{i-1,j-1} + dp_{i-1,j-2} + dp_{i-2,j-1} + 2dp_{i-2,j-2} + dp_{i-2,j-3}.$$

Ta xét cách tối ưu chuyển trạng thái này.

Gọi F_i là hàm sinh của dp_i . Có hai cách làm.

Cách 1. Từ truy hồi trên, ta có phương trình chuyển của hàm sinh:

$$F_i = (x^2 + x + 1)F_{i-1} + (x^3 + 2x^2 + x)F_{i-2}.$$

Áp dụng lũy thừa ma trận nhanh cho truy hồi này là đủ.

Cách này có thể có hằng số khá lớn, nên cần một số tối ưu hằng số. Độ phức tạp chủ yếu đến từ việc tính NTT, vì vậy ta xét cách giảm số lần dùng NTT. Khi nhân hai ma trận 2×2 , ta cần tổng cộng $8 \cdot 2$ lần DFT và 8 lần IDFT, tức 24 phép NTT. Trong đó một nửa các DFT bị tính lặp lại, nên có thể tiền xử lý DFT của hai ma trận để bỏ được 8 phép NTT. Ngoài ra, phép toán ma trận chỉ dùng nhân và cộng, mà trong biểu diễn theo giá trị điểm thì hai phép này vẫn tính được như thường; do đó ta có thể cộng xong toàn bộ một giá trị trong ma trận rồi mới thực hiện IDFT, tiết kiệm thêm 4 phép NTT. Như vậy hằng số được giảm một nửa.

Tương tự, trong lũy thừa nhanh có một nửa phép toán là bình phương một ma trận; với trường hợp này chỉ cần thực hiện 4 phép DFT.

Cách 2. Có thể dùng tư tưởng nhân đôi. Với F_{i+j} , ta xét cắt ở giữa. Nếu hai món ở giữa không bị buộc chung, hàm sinh số phương án là

$$F_i * F_j.$$

Nếu hai món ở giữa bị buộc thành một khối, tương tự chuyển trạng thái ban đầu, hàm sinh lúc này là

$$(x^3 + 2x^2 + x)F_{i-1} * F_{j-1}.$$

Vì thế có thể chuyển bằng nhân đôi; mỗi lần có thể chuyển từ F_{i-2}, F_{i-1}, F_i sang $F_{2i-2}, F_{2i-1}, F_{2i}$. Như vậy cũng chỉ cần tính $O(\log n)$ lần.

Độ phức tạp cuối cùng của cả hai cách đều là

$$O(m \log m \log n).$$

1003 Ink on Paper

Đây là một bài cây khung nhỏ nhất đơn giản. Đề bài có thể chuyển thành tìm cây khung nhỏ nhất trên một đồ thị đầy đủ; dùng trực tiếp Prim, độ phức tạp $O(n^2)$.

Kruskal với hằng số tốt cũng có thể qua, nhưng nếu nhị phân đáp án rồi tìm kiếm, độ phức tạp $O(n^2 \log \text{ans})$ có thể không qua.

1004 Counting Stars

Bài yêu cầu hỗ trợ ba thao tác:

1. tính tổng đoạn;
2. trừ $\text{lowbit}(a_i)$ trên một đoạn;
3. cộng 2^k trên một đoạn, với $2^k \leq a_i < 2^{k+1}$.

Gặp bài toán đoạn rất dễ nghĩ tới dùng cây đoạn để duy trì, nhưng hai thao tác sau không phải thao tác cơ bản mà cây đoạn hỗ trợ trực tiếp.

Giả sử chỉ có hai thao tác đầu, ta xét cách duy trì. Đây cũng là một bài khá kinh điển. Với những hàm làm giá trị giảm rất nhanh tới trạng thái ổn định như vậy, chẳng hạn lấy căn trên đoạn hoặc lấy hàm Euler trên đoạn, ta có thể cân nhắc sửa bạo lực. Phân tích độ phức tạp: trước hết hiểu rằng $x - \text{lowbit}(x)$ tương đương xóa bit 1 cuối cùng trong biểu diễn nhị phân của x . Có thể thấy mỗi số nhiều nhất bị thao tác $O(\log n)$ lần thì sẽ trở thành 0. Vì vậy tổng số lần sửa chỉ là $O(n \log n)$; tính cả độ phức tạp cây đoạn, ta có thể sửa trong $O(n \log^2 n)$ và hỏi tổng trong $O(n \log n)$. Khi cài đặt, để bỏ qua các đoạn đã trở thành 0 trong lúc sửa bạo lực, ta có thể duy trì trên cây đoạn một tag cho biết toàn bộ số trong đoạn có bằng 0 hay không; điều này có thể duy trì bằng đẩy nhãn của cây đoạn.

Tiếp theo xét cách thêm thao tác thứ ba. Theo ý tưởng phía trên, ta vẫn có thể nhìn thao tác này theo nhị phân. Thao tác thứ ba tương đương dịch bit 1 bên trái nhất của a_i sang trái một vị trí. Dễ thấy nó chỉ liên quan tới bit cao nhất của a_i , không liên quan tới các bit phía sau. Vì thế ta tách bit cao nhất của a_i và các vị trí còn lại ra để duy trì riêng. Khi đó thao tác thứ ba tương đương nhân đôi bit cao nhất trên đoạn; việc này cũng có thể duy trì bằng cây đoạn. Như vậy bài toán kết thúc; đây nên là một bài cấu trúc dữ liệu có độ khó tư duy không quá cao.

1005 Separated Number

Tổng đáp án không dễ tính trực tiếp, nên xét đóng góp của từng chữ số vào đáp án. Do cùng một số nhưng nằm ở vị trí khác nhau trong đoạn bị tách sẽ có đóng góp khác nhau, giả sử hiện đang xét chữ số thứ i của toàn bộ số, là d ($0 \leq d \leq 9$), và trọng số vị trí của nó trong đoạn tách chứa nó là 10^j . Khi đó đóng góp của nó là

$$d \cdot 10^j \cdot num,$$

trong đó num chỉ tổng số cách tách tạo ra trường hợp này. Ta xét cách tính num .

Vì lúc này chữ số thứ i của toàn bộ số đóng góp trọng số 10^j , nên sau vị trí thứ $i + j$ phải có một điểm tách. Giả sử n là tổng độ dài của số đầu vào, có hai trường hợp:

1. $i + j < n$. Khi đó ngoài điểm tách này ta còn cần chèn nhiều nhất $k - 2$ điểm tách, nên

$$num = \sum_{m=0}^{k-2} C_{n-j-2}^m.$$

2. $i + j = n$. Khi đó ta còn cần chèn nhiều nhất $k - 1$ điểm tách, nên

$$num = \sum_{m=0}^{k-1} C_{n-j-1}^m.$$

Ký hiệu

$$F(a, b) = \sum_{m=0}^b C_a^m.$$

Ta có

$$F(a, b) = 2F(a - 1, b) - C_{a-1}^b.$$

Vì vậy có thể tiền xử lý tất cả các giá trị $F(a, k - 1)$ và $F(a, k - 2)$, nhờ đó tính nhanh được num .

Với trường hợp $i + j = n$, cộng trực tiếp vào đáp án. Với trường hợp $i + j < n$, để ý rằng lúc này $10^j \cdot num$ chỉ phụ thuộc vào j , nên có thể dùng tổng tiền tố để tối ưu và tính nhanh kết quả.

1006 GCD Game

Đây là Nim thay áo: xem mỗi số như một đồng sỏi, số viên sỏi chính là số thừa số nguyên tố của số đó.

Chỉ cần dùng sàng tuyến tính tiền xử lý số lượng thừa số nguyên tố cho các số không vượt quá 10^7 .

1007 A Simple Problem

Dùng cây đoạn duy trì cộng đoạn và max trên đoạn.

Xét cách thống kê đáp án giữa hai đoạn: max hậu tố của đoạn trước và max tiền tố của đoạn sau đều đơn điệu. Có thể nhị phân trên dãy; mỗi lần chọn một đoạn có độ dài đúng bằng k . Nếu max của đoạn trước lớn hơn, đoạn tối ưu sẽ không nằm bên trái đoạn này; ngược lại, nó sẽ không nằm bên phải đoạn này.

Trực tiếp dùng cách này để gộp đáp án cho ta thuật toán $O(q \log^3 n)$. Nếu có thể nhị phân ngay trên cây đoạn, ta đạt $O(q \log^2 n)$.

Lời giải đúng như sau. Với tất cả truy vấn, k là hằng số. Chia dãy theo k , phần vượt quá n được đếm tới bội của k . Khi đó k số liên tiếp hoặc nằm trong một đoạn, hoặc nằm giữa hai đoạn.

Với mỗi đoạn, xây một cây đoạn; hình dạng cây đoạn của mọi đoạn hoàn toàn như nhau, nên phép nhị phân có thể làm trên cây đoạn, mỗi lần nhị phân tốn $O(\log n)$. Mỗi lần sửa chỉ cần tính lại đáp án cho $O(1)$ đoạn ở hai đầu.

Ngoài ra, xây một cây đoạn khác để duy trì đáp án của n/k đoạn; cây này cần hỗ trợ cộng đoạn, sửa điểm và truy vấn min trên đoạn.

Khi sửa, thực hiện cộng đoạn cho phần giữa và tính lại hai đầu. Khi hỏi, truy vấn min cho phần giữa; các phần ở hai đầu không đủ một đoạn thì dùng nhị phân để tính đáp án.

Tổng độ phức tạp là $O(q \log n)$.

1008 Square Card

Có thể thấy để một tấm thẻ hình vuông có khả năng được một hình tròn chứa hoàn toàn, quỹ tích tâm của nó nhất định là một hình tròn. Vì vậy bài toán chuyển thành tính tỉ lệ giữa diện tích giao của hai hình tròn và diện tích hình tròn thứ nhất. Tuy nhiên vẫn có một số chi tiết, chẳng hạn trong dữ liệu vùng thường không nhất thiết chứa được toàn bộ tấm thẻ, và các vị trí tương đối khác nhau của hai hình tròn cần được chia trường hợp; chi tiết cụ thể có thể xem trong lời giải chuẩn.

1009 Singing Superstar

Đây là một bài chuỗi đơn giản. Với mọi xâu con của xâu mẹ có độ dài không quá 30, tính băm chuỗi; với các xâu có cùng giá trị băm thì thống kê đáp án bằng bạo lực. Mỗi truy vấn chỉ cần tra trực tiếp đáp án tương ứng với giá trị băm.

Cũng có thể dùng Trie hoặc tự động cơ AC để qua bài này. Giới hạn dữ liệu khá rộng, nên độ phức tạp lý thuyết hơi kém hơn cũng vẫn có thể thông qua.

1010 Yinyang

Vì cả màu đen lẫn màu trắng đều phải liên thông, chu trình nếu xuất hiện chỉ có thể nằm trên vòng ngoài cùng của lưới.

Do $nm \leq 100$ và $\min(n, m) \leq 10$, không ngại giả sử $n \geq m$ rồi xét quy hoạch động. Khi DP, cần ghi lại màu và tính liên thông của m ô trước đó; số trạng thái như vậy sẽ không vượt quá 20000.

Khi chuyển, liệt kê màu của vị trí tiếp theo, không cho phép tạo chu trình, và xử lý riêng hai ô cuối cùng.